

1. Introduction

Reinforcement Learning from Human Feedback (RLHF) allows agents to learn from preference comparisons instead of relying only on hand-designed rewards.

In this project, we investigate how preference-based learning can be applied to simple control environments such as **CartPole-v1** and **Pendulum-v1**. We compare two approaches: **PPO-RLHF**, which first learns a reward model from preferences, and **DPO**, which directly optimizes the policy from preferred and rejected trajectories.

Our goal is to analyze how these methods behave as the amount of preference data changes, and to compare their performance against standard PPO baselines.

2. Background and related work

PPO-RLHF

PPO-RLHF uses preferences in two steps:

1. Learn a reward model from preference pairs.

$$R_{\phi}(\tau) = \sum_{t=0}^T r_{\phi}(s_t, a_t)$$

$$P_{\phi}(\tau^+ \succ \tau^-) = \sigma(R_{\phi}(\tau^+) - R_{\phi}(\tau^-))$$

$$\mathcal{L}_{RM}(\phi) = -\frac{1}{K} \sum_{i=1}^K \log \sigma(R_{\phi}(\tau_i^+) - R_{\phi}(\tau_i^-))$$

2. Optimize the policy with PPO using the learned reward.

$$\max_{\theta} \mathbb{E}_{\tau \sim \pi_{\theta}} \left[\sum_{t=0}^T r_{\phi}(s_t, a_t) \right]$$

Direct Preference Optimization

DPO directly trains the policy from preferences, without explicitly learning a reward model.

$$\mathcal{L}_{DPO}(\theta) = -\frac{1}{K} \sum_{i=1}^K \log \sigma \left(\beta \log \frac{\pi_{\theta}(\tau_i^+)}{\pi_{\text{ref}}(\tau_i^+)} - \beta \log \frac{\pi_{\theta}(\tau_i^-)}{\pi_{\text{ref}}(\tau_i^-)} \right)$$

3. Methods

Step 1: Train two policies

CartPole-v1 + Pendulum-v1

Expert policy π_1

PPO trained to convergence, best mean reward

Mediocre policy π_2

Early PPO checkpoint, half max reward with low variance

τ_1 trajectories

Sequences of state, action, reward

τ_2 trajectories

Sequences of state, action, reward

Step 3: Build preference dataset

Preference dataset

K pairs $(\tau_1, \tau_2) \in \{500, 1000, 2000, 10000\}$ using Bradley-Terry labelling :

$$P(\tau_1 \succ \tau_2) = \frac{\exp(R_1)}{\exp(R_1) + \exp(R_2)}$$

Step 4: Run RLHF algorithms

PPO-RLHF

2-step approach

- Train reward model R_{ϕ}**
 - MLP learns to score each pair
 - Trained on K preference pairs until convergence
 - Freeze weights after training

- Policy fine-tuning**
 - Standard PPO using R_{ϕ} as reward signal
 - Policy is initialized with π_2 , our mediocre policy

$\pi_{\text{PPO-RLHF}}$

Fine-tuned policy

DPO

Direct 1-step approach

- Direct optimisation of policy π**
- π optimised directly on K preference pairs
 - Reference policy π_{ref} anchors updates via log-ratio penalty
 - β controls how far π can drift from π_{ref}
 - π_{ref} is initialized with π_2 , our mediocre policy

π_{DPO}

Fine-tuned policy

Step 5: Evaluate and compare

Evaluation

Avg. cumulative reward across 20 seeds, vary K $\in \{500, 1000, 2000, 10000\}$, compare PPO-RLHF vs DPO on CartPole-v1 and Pendulum-v1

4. Results

PPO Training Curves for Policy Checkpoint Selection (CartPole-v1 and Pendulum-v1)

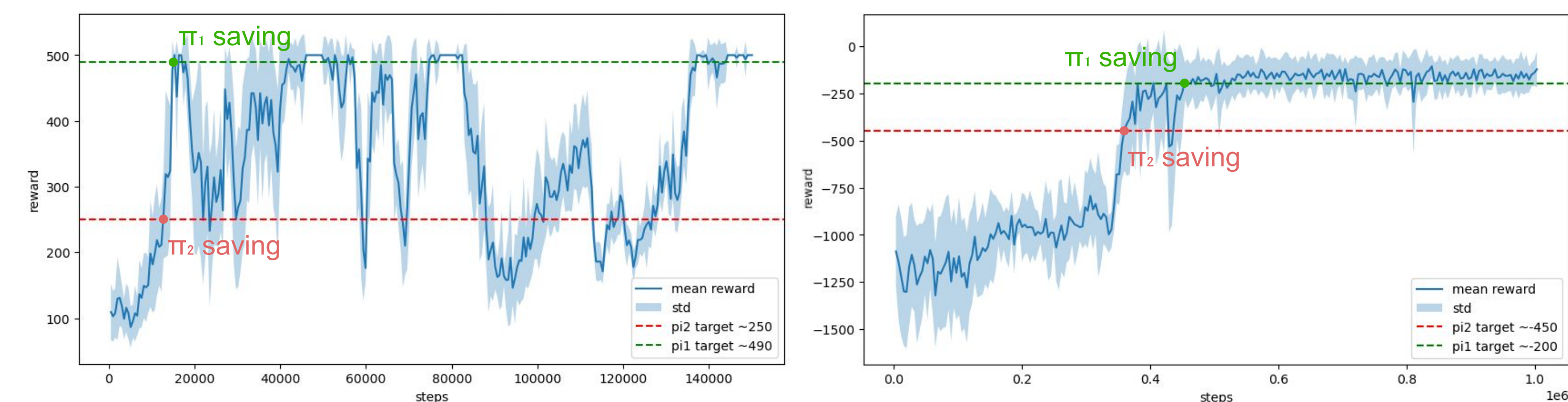


Figure 1. Training curves for PPO on CartPole-v1 (left) and Pendulum-v1 (right). Dashed lines indicate the target mean returns used to select the π_1 (expert, green) and π_2 (mediocre, red) checkpoints.

Reward Model: Predicted vs. True Episode Return

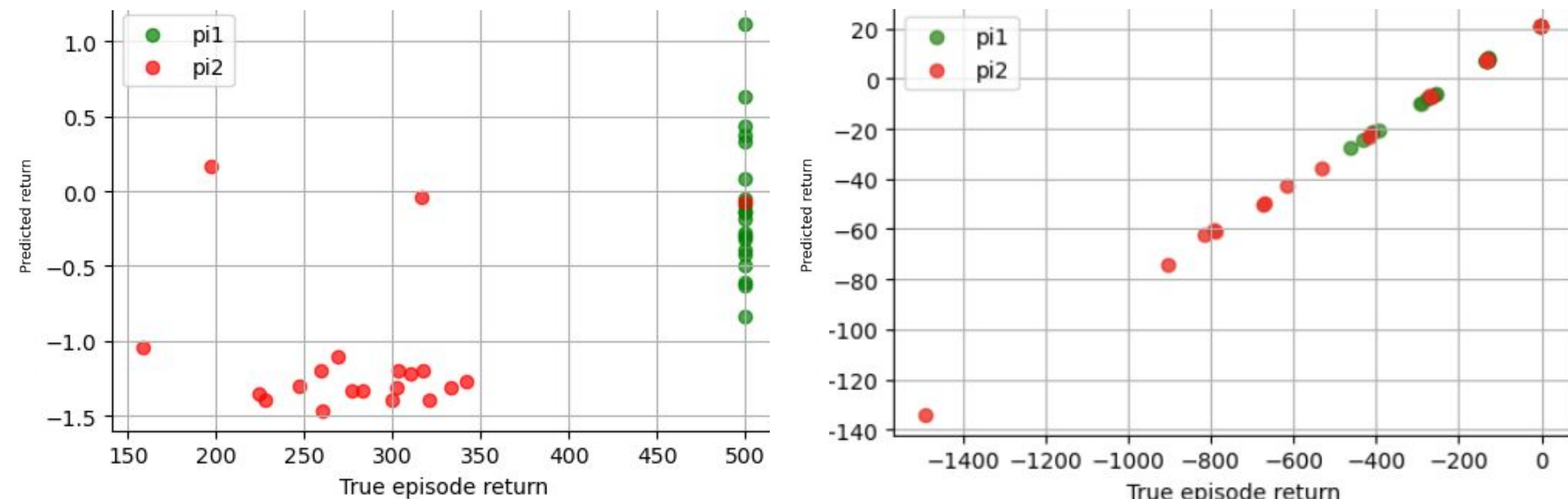


Figure 2. Predicted vs. true episode return for the learned reward model on CartPole-v1 (left) and Pendulum-v1 (right), trained on K=5000 preference pairs. Each point is one episode; green: π_1 (expert), red: π_2 (mediocre).

True Episode Reward During Fine-Tuning (DPO vs. PPO-RLHF)

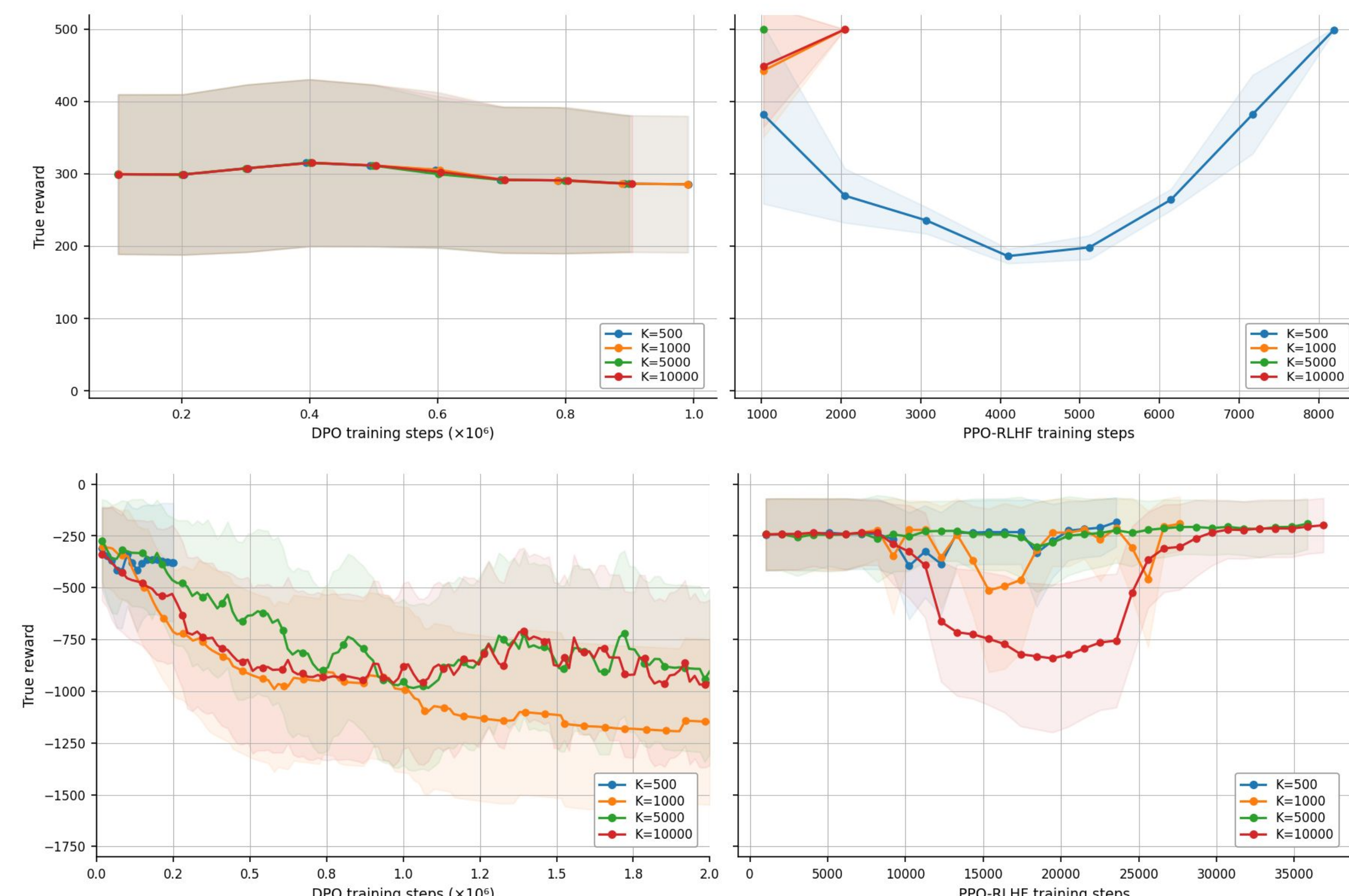


Figure 3. True episode reward during fine-tuning on CartPole-v1 (top) and Pendulum-v1 (bottom), using DPO (left) and PPO-RLHF (right), across preference dataset sizes K. Shaded regions indicate standard deviation across seeds.

Final Policy Performance vs. Preference Dataset Size (DPO vs. PPO-RLHF)

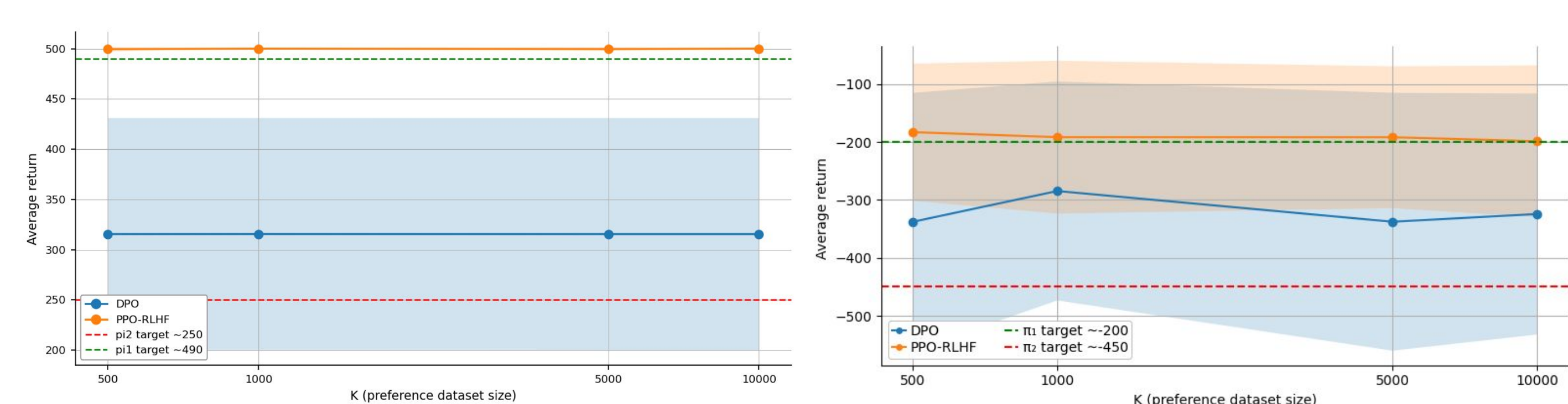


Figure 4. Average true episode return at the end of fine-tuning as a function of preference dataset size K, on CartPole-v1 (left) and Pendulum-v1 (right). Orange: PPO-RLHF, blue: DPO. Shaded regions indicate standard deviation across seeds. Dashed lines mark the π_1 (expert) and π_2 (mediocre) checkpoint returns for reference.

5. Discussion and conclusion

- As seen in **Figure 1**, PPO is able to learn an expert policy for both environments, although it takes an order of magnitude more training steps for Pendulum-v1.
- Figure 2** allows us to validate the reward model for PPO-RLHF. It learns a useful reward function, that is predictive of the true reward and that can easily be used to separate good from bad trajectories.
- Although DPO learns a policy slightly better than π_2 , it still proves to be less stable than PPO-RLHF: in **Figure 3**, we see that it fails to reach best reward on all dataset sizes, quickly collapsing to a worse policy than π_2 .
- Figure 4** shows no improvement of performance with preference dataset size, and even a decrease in performance for DPO on Pendulum-v1.
- However, increasing dataset size can still have a positive effect, especially for PPO-RLHF. In **Figure 3**, we see that it allows PPO-RLHF to reach expert level with less training steps on CartPole-v1. But this cannot be concluded for Pendulum-v1, where the effect seems to be the opposite, likely due to the slower and less stable training of PPO on Pendulum-v1.
- We conclude that PPO-RLHF remains more stable and performant than DPO when applied to control tasks, where the action space, state space, and reward structure differ substantially from the LLM setting in which DPO was originally proposed.

References

- [1] Christiano et al., Deep Reinforcement Learning from Human Preferences, 2017.
- [2] Schulman et al., Proximal Policy Optimization Algorithms, 2017.
- [3] Rafailov et al., Direct Preference Optimization: Your Language Model is Secretly a Reward Model, 2023.
- [4] Zheng et al., Secrets of RLHF in Large Language Models Part I: PPO, 2023.

